Handoff 02: ENG-02 migrations and XP/badge ledgers

Date: 2026-10-25 (America/Toronto)
Written by: Claude Code (implementer)
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: pull request #7 open, not merged, not deployed

> **How to use this file:** upload it with `build.md` (and handoff 01 if the chat is new). It summarizes evidence; the pull request diff and CI results are the source of truth (`build.md` section 14).

Starter prompt for the chat assistant

```
You are reviewing Python Academy pull request #7 (ENG-02: database
migrations and XP/badge ledgers). I'm attaching build.md and this handoff.
Check the plan against the acceptance criteria below. Focus on: could
the first deploy damage or lose existing production data; is anything
awarded twice or wrongly; does anything learners see change; what is
not yet verified. Explain at a first-year computer engineering level,
and only call something tested if this handoff shows the evidence.
```

1. Where we are

Stage	What	Status
Plan	`docs/plan/course-catalog.md`	**Merged** (PR #6)
1. ENG-01	Content format and result states	**Merged** (PR #5) and **verified live**: `/health` ok, `/api/lessons` identical to the original, `/api/check` returns `result` (`passed` and `unsupported` checked)
2. ENG-02	Alembic migrations, XP and badge ledgers, backfill	**in_review**: PR #7
3. Catalog UI	Catalog, course, and lesson pages with unit sidebar	Not started
4. Goals and recommendations	Goal card and rule-based recommendation function	Not started

Rules for every stage: one branch and PR; exact test commands and results; never merge or deploy; no secrets or paid services; no change to age rules or account policy; never run learner code on the server; stop for review.

2. Handoff record (build.md template)

Field	Value
Task / issue	ENG-02: schema migrations, XP and badge award ledgers, backfill (plan sections 6 and 7.4)
Human goal	Make database changes safe and repeatable, and record XP and badges so they can never be awarded twice, without losing existing accounts
Relevant build.md sections	10 (badges), 11 (proposed data changes), 12 (per-feature and broader-release gates)
Repository and branch	`torontojohn/python-academy`, branch `eng-02/migrations`
Base commit / code commit	Base `4fa9b21` (`main` after PR #5) / code `71b4b30`
Pull request	https://github.com/torontojohn/python-academy/pull/7
Deployment status	Not requested. Not merged, not deployed

What changed, in plain terms

- **Migrations replace `create_all`.** A migration is a numbered, versioned script that changes the database. The app runs any new ones when it starts.
- **Adopting the live database safely.** Production's tables were made before migrations existed. On first start, the app checks that the four existing tables have exactly the expected columns. If they do, it records them as migration `0001` without touching any data, then applies the new migrations. If they don't, it stops and changes nothing; Render keeps the old version running. All of this happens in one transaction.
- `0002` adds two new tables, `xp_awards` and `badge_awards`, and a new optional `result` column on `attempts`. It only adds things, so the old version keeps working during the deploy.
- `0003` **backfill.** Fills the new tables from existing passing attempts, using the same rule `/api/progress` already uses. Running it again adds nothing.
- **Awards on new passes.** A passed challenge now writes 100 XP and the lesson's badge in the same transaction as the attempt. A database unique key plus `INSERT … ON CONFLICT DO NOTHING` means retries or two simultaneous requests can't award twice. A new version of a task or badge does not award again.

Files changed

File	Change
`backend/app/db.py`	New. Engine and tables (models moved unchanged from `main.py`), new award tables, `insert_once` helper
`backend/app/migrate.py`	New. Adopt-or-upgrade at startup, PostgreSQL advisory lock
`backend/app/awards.py`	New. `award_challenge`, `total_xp`, `badges`
`backend/migrations/` + `backend/alembic.ini`	New. Alembic environment and migrations `0001`–`0003`
`backend/app/main.py`	Uses the above; stores `result`; awards on pass; logs the schema version at startup
`backend/tests/test_migrations.py`	New. 7 tests, each on SQLite and PostgreSQL
`backend/tests/test_academy.py`	2 new API tests for awarding
`.github/workflows/ci.yml`	Throwaway PostgreSQL 16 service for the backend job
`backend/requirements.txt`	`alembic`, `Mako`, `MarkupSafe` (exact pins)
`README.md`, `DEPLOYMENT.md`, `build.md`	How migrations work, what the first deploy does, source locations

Acceptance criteria

- Existing accounts survive. **Met in tests and rehearsal**: every row in all four original tables is unchanged after migration, on SQLite and PostgreSQL. **Not yet verified on production.**
- Baseline first. **Met**: `0001` matches the original tables exactly (automated schema comparison).
- XP and badge ledgers with unique keys. **Met**.
- Backfill from existing attempts. **Met**: matches `/api/progress` rules and is idempotent.
- No double awards from retries, concurrency, or new versions. **Met**.
- Nothing learners see changes. **Met**: 15 browser tests and a real-stack run.

Tests run

Local: Python 3.12.14, Node 24.19.0, PostgreSQL 16.2 (throwaway local server via `pgserver`, dev-only).

Command	Before (`main`)	After (`71b4b30`)
`cd backend && python -m pytest -q` (SQLite only)	71 passed	80 passed, 7 skipped (the PostgreSQL variants)
`cd backend && TEST_POSTGRES_URL=… python -m pytest -q`	—	**87 passed**
`cd frontend && npm run build`	succeeded	**succeeded**
`cd frontend && npm run test:e2e`	15 passed	**15 passed**

Command	Before (`main`)	After (`71b4b30`)
GitHub Actions on PR #7 (with PostgreSQL service)	—	**87 backend passed; 15 browser passed**

New migration tests (each on SQLite and PostgreSQL):

- A fresh database built by migrations matches the models exactly.
- `0001` matches the original tables exactly.
- An old database with realistic learner data is adopted with every row unchanged.
- The backfill gives the right awards and adds nothing when re-run.
- A database with unexpected tables is refused untouched.
- Downgrade then upgrade keeps learner data and rebuilds awards.
- A duplicate award insert is ignored.

Production rehearsal on PostgreSQL with the real old code:

1. The code currently on `main` created the tables and saved attempts through its own API. Progress was `['hello']`; a legacy arithmetic attempt correctly did not count.
2. The new code started on that database: migration `0003`, all 4 attempts kept, backfill = 100 XP and Story spark for `hello`, matching progress.
3. The old code then ran against the migrated database and still saved an attempt (HTTP 200) with correct progress. This matters because during a deploy the old version keeps serving briefly.

Real stack on PostgreSQL: uvicorn plus `next start`. The log showed `Database schema is at migration 0003`, `/health` was ok, and all five lessons passed the real check in Chromium, plus the 375px layout.

Bug caught by PostgreSQL testing: on PostgreSQL the driver reports an unknown row count for `ON CONFLICT DO NOTHING`, so the code couldn't tell whether an award was inserted. Fixed by using `RETURNING`.

Mocks and unverified behaviour

- Browser tests mock account and check responses; Python execution is real.
- **Not verified:** the first deploy against Render's production PostgreSQL. Its version isn't recorded in the repo; tests use 16.

Known limitations

- **Deploy-overlap gap.** A passing attempt saved by the old version during the few seconds both versions run gets no ledger row, because the backfill already ran. Harmless now, because nothing reads the ledgers yet. Stage 3 should re-run the same idempotent backfill just before the UI starts reading them.
- `/api/progress` still derives progress from attempts; the ledgers become the source of truth in stage 3.
- No endpoint exposes XP or badges yet.

After merging: checking the live site

1. Render → **python-academy-api** → **Events**: wait for "Deploy live". If it shows "Deploy failed", the old version is still serving; send the log to Claude Code.
2. Render → **python-academy-api** → **Logs**: look for `Database schema is at migration 0003`.
3. https://python-academy-dji8.onrender.com/health shows `{"status":"ok"}`.
4. Sign in and check your dashboard and lessons load; in preview, pass a challenge.

Next concrete action

Maintainer reviews PR #7. After merge and a live check, start stage 3 (catalog UI) on a new branch, beginning with the idempotent backfill re-run and read endpoints for the catalog and awards.